「樽前作剧莫相笑，我死诸君思此狂」

Wi. Zhimu Yang

Hydrangea Academy of Magic, HAM

Magnolia Institute of Science, MIS.HAM

计算理论导引（1）

计算理论导引（1）
- 一、Automata(自动机) & Languages(语言)
  - 1. Finite Automata(有穷自动机) / Finite State Machine(有穷状态机)
    - Definition of Finite Automata(有穷自动机的定义)
    - Computation of Finite Automata(有穷自动机的计算)
    - Regular Operations(正则运算)
    - Nondeterminism(不确定性)
      - Definition of Nondeterministic Finite Automata(非确定型有穷自动机的定义)
      - Computation of Nondeterministic Finite Automata(非确定型有穷自动机的计算)
    - Equivalence of NFAs and DFAs(NFA与DFA的等价性)
    - Regular Expressions(正则表达式)
    - Equivalence of Regular Expressions and Finite Automata(正则表达式与有穷自动机的等价性)
    - Generalized Nondeterministic Finite Automata(广义非确定型有穷自动机)
      - Definition ofGeneralized Nondeterministic Finite Automata(广义非确定型有穷自动机的定义)
      - Computation of Generalized Nondeterministic Finite Automata(广义非确定型有穷自动机的计算)
    - Nonregular Languages(非正则语言)
      - The Pumping Lemma for Regular Languages(正则语言的泵引理)

> *Type:*
>
> *~~Tutorial~~ / Note*
>
> *For:*
>
> *Amateurs / ~~Professionals~~*
>
> *Reference:*

> *[1] Michael Sipser . 《Introduction to the Theory of Computation》. 机械工业出版社, 2015*

什么是计算？什么是计算机？我们尝试通过数学理论的描述去回答这个问题。

一、Automata(自动机) & Languages(语言)

1. Finite Automata(有穷自动机) / Finite State Machine(有穷状态机)

Definition of Finite Automata(有穷自动机的定义)

对于一个简单的计算模型我们可以描述为一个有穷自动机。

> *Definition*
>
> ***Finite Automata(有穷自动机)***
>
> *一个有穷状态机 M 可以描述为5元组（Q，Σ，δ，q_0，F）*
>
> *Q：State(状态集)，有穷集合，所有状态集合*
>
> *Σ：Alphabet(输入字母表)，有穷集合，所有允许的输入符号集*
>
> *δ：Transition Function(转移函数)，定义域为有序对集合 $Q \times \Sigma$，值域为 Q，即 $\delta : Q \times \Sigma \to Q$*
>
> $\delta(x, a) = y$ *：描述当处于状态 x、接受到输入 a时，转移到状态 y*
>
> q_0 *：Start State(起始状态)，$q_0 \in Q$*
>
> *F：Final State(终止状态)/Accept State(接受状态)，$F \in Q$*

若集合 A 是机器 M 所接受的全部字符串集，称 A 为机器 M 的 Language(语言)，记作 $L(M) = A$ ，说机器 M 识别/接受语言 A 。一台机器可以接受若干字符串，但永远只能识别一种语言，当它不接受任何字符串时，它仍然识别一种语言——空语言 $\emptyset$ 。

Computation of Finite Automata(有穷自动机的计算)

有穷自动机的运行计算过程可以概括如下：

> *Definition*
>
> *有穷自动机* $M=(Q,\Sigma,\delta,q_0,F)$，*字符串* $w=w_1w_2w_3\cdots w_n$ *且其中任一* $w_i\in\Sigma$。*若存在* Q *中的状态序列* $r_0,r_1,r_2,\cdots,r_n$ ，*满足：*
>
> 1. $r_0=q_0$，*从起始状态开始*
> 2. $\delta(r_i,w_{i+1})=r_{i+1}$，*按照转移函数从一个状态到另一个状态*
> 3. $r_n\in F$，*机器处在终止状态*
>
> *则机器* M *接受/识别字符串* w。

若语言 A={w}，即语言 A 被机器 M识别，则称 A 为Regular Language(正则语言)。

Regular Operations(正则运算)

正则语言具有一些有趣的性质。

> *Definition*
>
> A、B *两个正则语言，定义正则运算：*
>
> - ***Union(并)***：$A\cup B$ = {$x|x\in A$ *or* $x\in B$}，A、B*的字符串合并在同一个语言中*
> - ***Concatenation(连接)***：$A\circ B$ = {$xy|x\in A$ *and* $y\in B$}，A*的一个字符串和B的一个字符串拼接为新字符串组成新语言*
> - ***Star(星)***：A* = {$x_1x_2\cdots x_k|k\geq 0$ *and* $x_i\in A$}，*一个语言中的任意(包含0)字符串拼接在一起得到新语言*

> *Theorem*
>
> *The class of regular languages is closed under the union、concatenation operation.*
>
> *正则运算在并运算、连接运算下均封闭。*
>
> *即：* A_1 、A_2 *是正则语言，则* $A_1\cup A_2$ 、$A_1\circ A_2$ *也是正则语言。*

Nondeterminism(不确定性)

Deterministic Finite Automata, DFA(确定型有穷自动机)：机器对于给定的状态和输入，下一状态确定。

Nondeterministic Finite Automata, NFA(非确定型有穷自动机)：机器对于给定的状态和输入，下一状态有种若干选择。

Definition of Nondeterministic Finite Automata(非确定型有穷自动机的定义)

> *Definition*
>
> ***Nondeterministic Finite Automata(非确定型有穷自动机)**：一个非确定型有穷状态机 N 可以描述为5元组（Q，Σ，δ，q_0，F）*
>
> *Q：State(状态集)，有穷集合，所有状态集合*
>
> *Σ：Alphabet(输入字母表)，有穷集合，所有允许的输入符号集*
>
> *δ：Transition Function(转移函数)，定义域为有序对集合 $Q \times \Sigma_\epsilon$，值域为 $\mathcal{P}(Q)$，即 $\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$*
>
> > *Σ_ϵ：$\Sigma \cup \{\epsilon\}$*
> >
> > *$\mathcal{P}(Q)$：Q 的 power set(幂集)，即 Q 的所有子集组成的集合*
>
> *q_0：Start State(起始状态)，$q_0 \in Q$*
>
> *F：Final State(终止状态)/Accept State(接受状态)，$F \in Q$*

Computation of Nondeterministic Finite Automata(非确定型有穷自动机的计算)

> *Definition*
>
> *非确定型有穷自动机 $N = (Q, \Sigma, \delta, q_0, F)$，字符串 $w \in \Sigma$，若 w 能够表示为 $w = y_1 y_2 y_3 \cdots y_m$ 且 $y_i \in \Sigma_\epsilon$，并且存在 Q 中的状态序列 $r_0, r_1, r_2, \cdots, r_n$，满足：*

> 1. $r_0 = q_0$，*从起始状态开始*
> 2. $r_{i+1} \in (r_i, y_{i+1})$，$r_{i+1}$ *是当* N *在状态* r_i*、接受到输入* y_{i+1} *时允许的下一个状态*
> 3. $r_m \in F$，*机器处在终止状态*
>
> *则机器* N *接受/识别字符串* w*。*

Equivalence of NFAs and DFAs(NFA与DFA的等价性)

如果两台机器识别同一种语言，我们称两台机器 equivalent(等价)。

> *Theorem*
>
> *Every nondeterministic finite automata has an equivalent deterministic finite automata.*
>
> *每台非确定型有穷自动机都等价于某一台确定型有穷自动机。*

> *Corollary*
>
> *An language is regular if and only if some nondeterministic finite automata recognizes it.*
>
> *一个语言是正则的，当且仅当有一台非确定性有穷自动机它。*

Regular Expressions(正则表达式)

在算术中，我们使用运算符来构造算术表达式，同理我们可以使用正则运算符构造正则表达式。

> *Definition*
>
> *我们称* R*为一个Regular Expression(正则表达式)，当* R*满足：*
>
> 1. a ，$a \in \Sigma$
> 2. ϵ，

> *3.* $\emptyset$，
>
> *4.* $(R_1 \cup R_2)$，R_1、R_2*为正则表达式*
>
> *5.* $(R_1 \circ R_2)$，R_1、R_2*为正则表达式*
>
> *6.* (R_1^*)，R_1*为正则表达式*

Equivalence of Regular Expressions and Finite Automata(正则表达式与有穷自动机的等价性)

正则语言是被有穷自动机识别的语言，自然而然地，任何正则表达式都应该能转换成能识别它所描述语言的有穷自动机，反之亦然。

> *Theorem*
>
> *A language is regular if and only if some regular expression describes it.*
>
> *一个语言是正则的，当且仅当可以用正则表达式表述它。*
>
> - *Lemma：If a language is described by a regular expression, then it is regular.*
> *如果一个语言可以用正则表达式描述，则它是正则的。*
> - *Lemma：If a language is regular, then it is described by a regular expression.*
> *如果一个语言是正则的，则它可以用正则表达式描述。*

Generalized Nondeterministic Finite Automata(广义非确定型有穷自动机)

Definition ofGeneralized Nondeterministic Finite Automata(广义非确定型有穷自动机的定义)

> *Definition*
>
> ***Generalized Nondeterministic Finite Automata(广义非确定型有穷自动机)，GNFA：****一个广义非确定型有穷自动机* G *可以描述为5元组（Q，Σ，δ，*q_{start}*，*q_{accept}*）*
>
> *Q：State(状态集)，有穷集合，所有状态集合*

> Σ：*Alphabet(输入字母表)，有穷集合，所有允许的输入符号集*
>
> δ：*Transition Function(转移函数)，定义域为有序对集合*$(Q-\{q_{accept}\})\times(Q-\{q_{start}\})$*，值域为*$R$*，即*$\delta:(Q-\{q_{accept}\})\times(Q-\{q_{start}\})\to\mathcal{R}$
>
> R：Σ*上全体正则表达式组成的集合*
>
> $\delta(q_i,q_j)=R$：*状态转移图中，从状态*q_i*到状态*q_j*的箭头以正则表达式*R*作为它的标记*
>
> $(Q-\{q_{accept}\})\times(Q-\{q_{start}\})$：*状态转移图中，除了*$q_{accept}$*没有射出的箭头、*$q_{start}$*没有射入的箭头，每一个状态到另一个状态都有一个箭头连接*
>
> q_{start} ：*Start State(起始状态)*
>
> q_{accept}：*Final State(终止状态)/Accept State(接受状态)*

Computation of Generalized Nondeterministic Finite Automata(广义非确定型有穷自动机的计算)

> *Definition*
>
> *广义非确定型有穷自动机*$G=(Q,\Sigma,\delta,q_{start},q_{accept})$*，若 字符串*$w$ *能够表示为*$w=w_1w_2w_3\cdots w_k$ *且*$w_i\in\Sigma^*$ *，并且存在 状态序列*$q_0,q_1,q_2\cdots,q_k$ *，满足：*
>
> 1. $q_0=q_{start}$*为起始状态*
> 2. $q_k=q_{accept}$*为终止状态*
> 3. *对每一个*i*，*$w_i\in L(R_i)$*，其中*$R_i=\delta(q_{i-1},q_i)$*，即*R_i*是从*q_{i-1}*到*q_i*的箭头上的表达式*
>
> *则机器*G *接受/识别字符串*w*。*

Nonregular Languages(非正则语言)

要了解有穷自动机的能力，我们应当知道有穷自动机的局限，也就是一些不能被有穷自动机识别的非正则语言。

The Pumping Lemma for Regular Languages(正则语言的泵引理)

泵引理指出，所有正则语言都具有一种特殊的性质，如果能够证明一个语言没有这一性质，则可以保证它不是正则的。

> *Theorem*
>
> ***Pumping Lemma(泵引理)***
>
> *If A is a regular language, then there is a number p (the pumping length) where if s is any string in A of length at least p, then s may be divided into three pieces, $s = xyz$, satisfying the following conditions:*
>
> 1. *for each $i \geq 0, xy^iz \in A$*
> 2. *$|y| > 0$*
> 3. *$|xy| \leq p$*
>
> *where $|s|$ represents the length of string s, y^i means that i copies of y are concatenated together, y^0 equals ϵ.*
>
> *若 A是一个正则语言，则存在一个数 p（泵长度），使得 A中任意长度不小于p的字符串 s可被分为3段，即 $s = xyz$，满足以下条件：*
>
> 1. *对每一个 $i \geq 0, xy^iz \in A$*
> 2. *$|y| > 0$*
> 3. *$|xy| \leq p$*
>
> *其中 $|s|$是字符串 s的长度，y^i 是 i 个 y相连，y^0 等于 ϵ。*

现在我们来回答先前的问题，什么是计算？什么是计算机？

计算是一个过程，即对于输入，给出输出结果。

计算机是一个数学模型，即对于输入指令，自动执行计算的过程，给出执行结果。

也许会觉得两者相同？其实存在着本质的差别，计算是一个固定的函数，是对数进行处理，“计算”不需要明白指令，输入什么数，就按照固定的数学表达式给出固定的结果，也即数学表达式是已知的；计算机是一个数学模型，它不是对数进行处理，而是对“计算”这个过程进行处理，“计算机”需要读懂指令——也就是我们计算理论中称为“语言”的表达式，也即

数学表达式是未知的，需要从语言中读出数学表达式。也因此，语言是一种描述性的指令，计算机自此被视为是一种描述和处理语言的数学模型。

$$\text{计算：数} \rightarrow \text{结果}$$

$$\text{计算机：语言} \rightarrow \text{结果}$$

在这一次我们介绍了有穷自动机和正则表达式两种描述语言的模型，当然它们其实都不够完备，还有一些语言它们不能够描述，我们在追求通用计算机的路上自然希望计算机能够处理较为全面的指令语言，所以我们会在下一文中延续介绍更多更全面更强大的计算数学模型。